

Lenguajes y Computación

Universidad Nacional Experimental de Guayana

Vicerrectorado Académico

Coordinación General de Pregrado

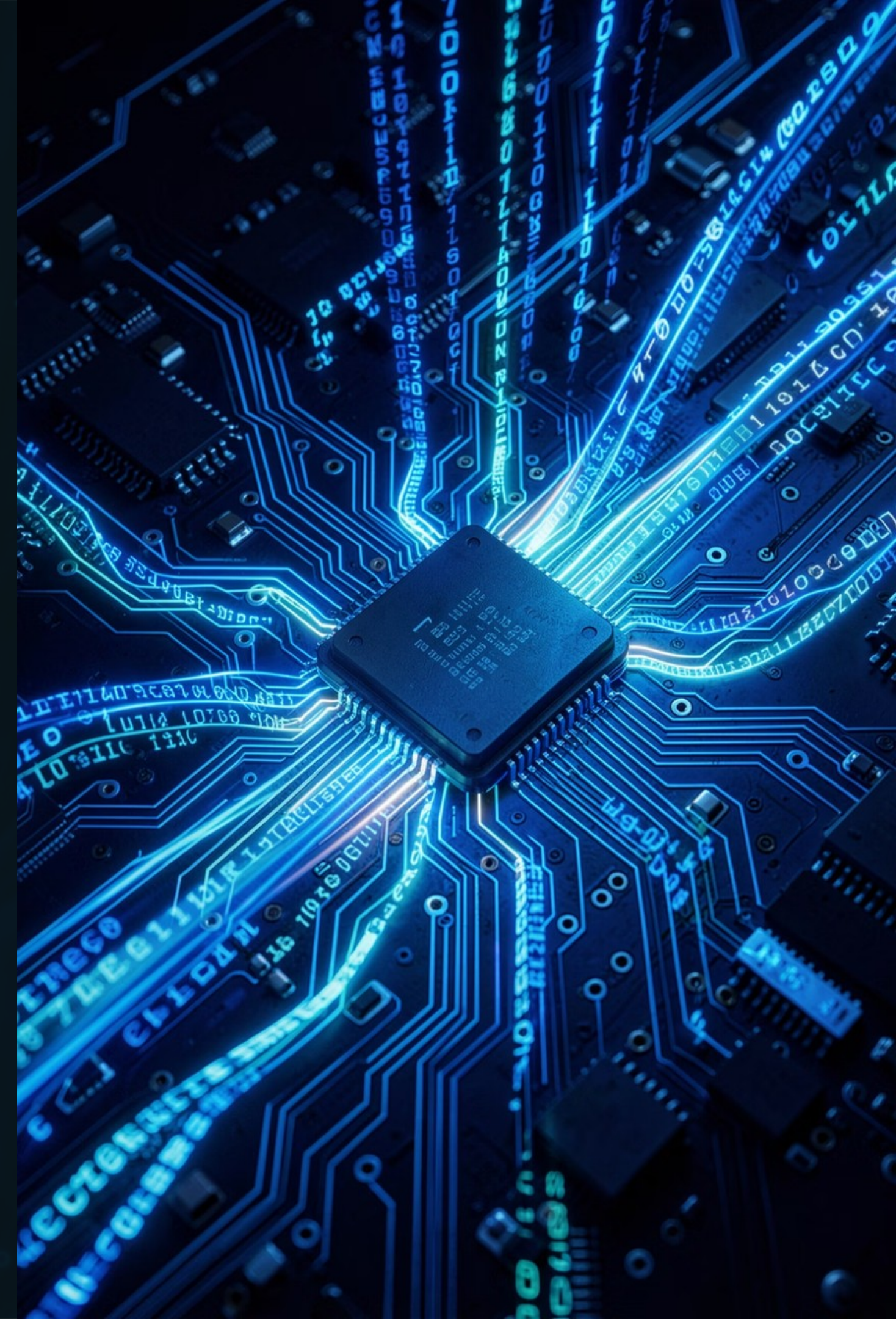
Proyecto de Carrera: Ingeniería Informática

Periodo 2026-II

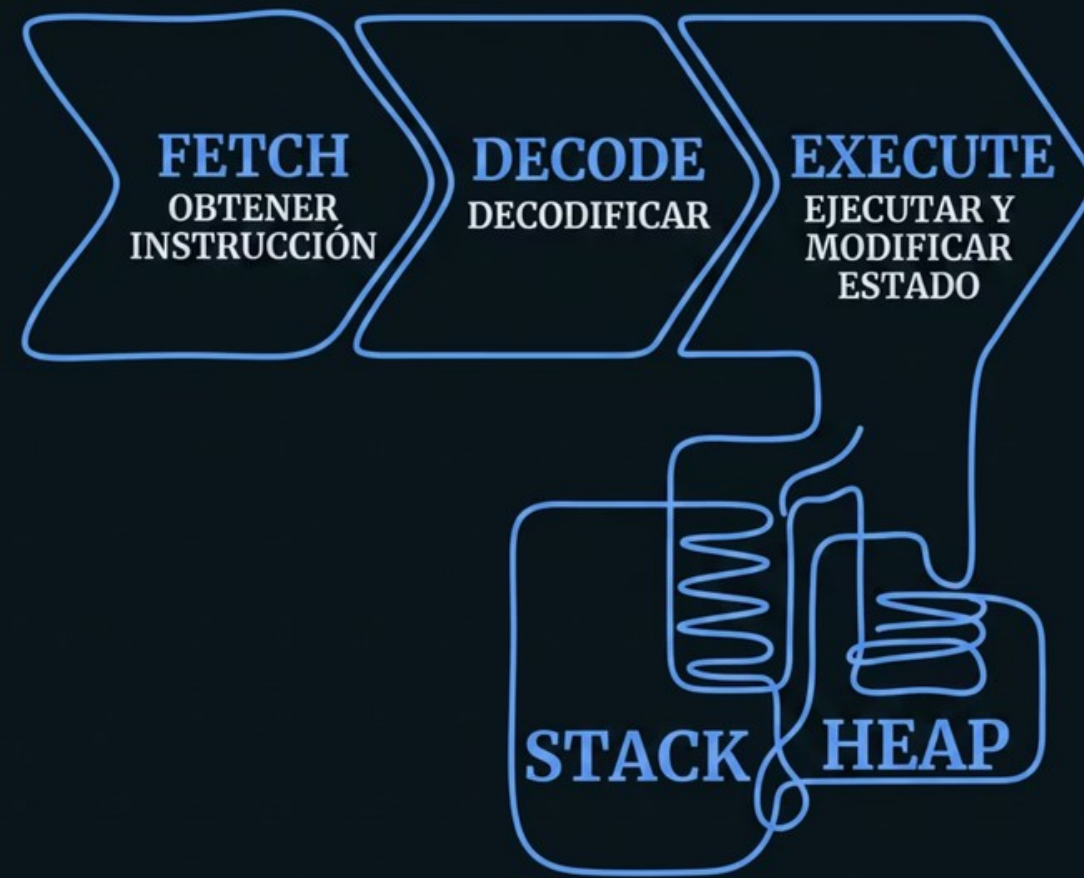
Unidad Curricular: Lenguaje y Compiladores

Integrantes: Iván Hernández · Daniel Valenzuela · Miguel

Barrios · Miguel Gómez



Paradigma Imperativo / Estructural



Reflejo directo de la arquitectura Von Neumann. La CPU opera mediante el ciclo **fetch-decode-execute**, modificando el estado de memoria paso a paso.

Teorema de Böhm-Jacopini

Cualquier función computable requiere solo tres estructuras: secuencia, selección e iteración.

Aliasing y Explosión de Estados

Múltiples punteros acceden al mismo espacio físico en Stack y Heap, generando efectos secundarios impredecibles.

PARADIGMA 2

Paradigma Orientado a Objetos

Unifica datos y algoritmos en entidades autónomas bajo los principios de **alta cohesión** y **bajo acoplamiento**.



Encapsulamiento

Guardián de invariantes. Principio *Tell, Don't Ask*: el objeto ejecuta operaciones según su lógica interna.



Polimorfismo Dinámico

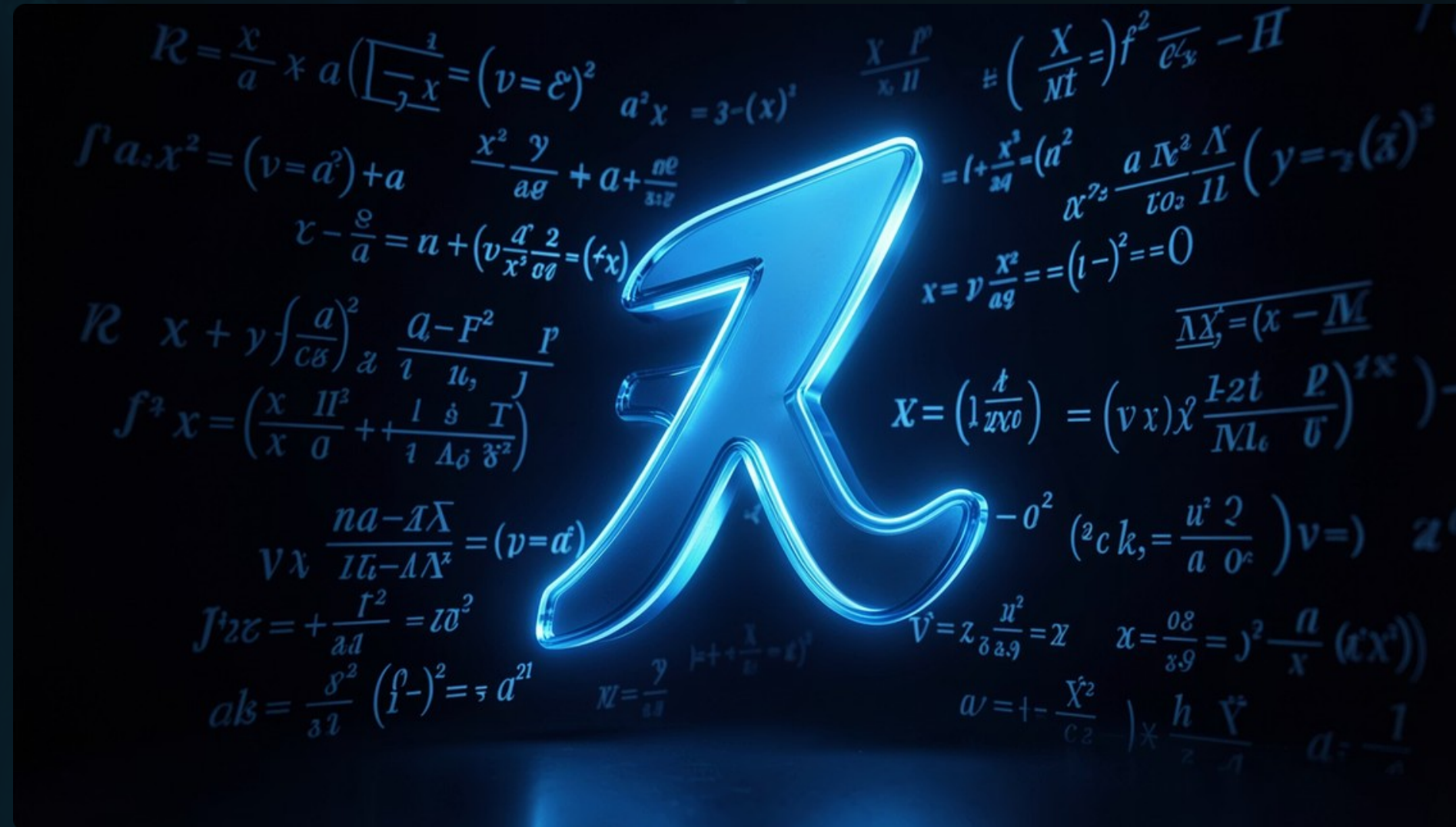
Resuelto en tiempo de ejecución mediante la **VTABLE** (tabla de métodos virtuales).



Composición vs Herencia

Se prioriza la composición para evitar el antipatrón de la *clase base frágil* y cumplir el Principio de Sustitución de Liskov.

Paradigma Funcional



Fundamentado en el **cálculo lambda** de Alonzo Church. La programación como evaluación de funciones matemáticas puras, sin estado mutable.

→ Inmutabilidad y Compartición Estructural
Valores de solo lectura (*bindings*) y árboles persistentes para optimizar memoria.

→ Orden Superior y Transparencia Referencial
Map, Filter y Reduce. Toda expresión puede sustituirse por su valor sin alterar el programa.

→ Evaluación Perezosa (*Lazy*)
Los cálculos se posponen en *thunks* hasta ser necesarios, permitiendo streams infinitos.

Paradigma Lógico / Declarativo

El programador define **qué computar**, no cómo. El sistema opera bajo la Hipótesis del Mundo Cerrado.



1

Cálculo de Predicados

Hechos axiomáticos y reglas condicionales (Cláusulas de Horn).

2

Unificación y Resolución SLD

Emparejamiento sintáctico bidireccional con búsqueda en profundidad y backtracking automático.

3

Ecuación de Kowalski

Algoritmo = Lógica + Control. La lógica es del programador; el control, del motor.

Paradigma Concurrente / Basado en Actores

Ante el fallo de los hilos imperativos, el **Modelo de Actores** impone aislamiento absoluto y comunicación asíncrona.



Actor: Estado + Mailbox FIFO

Un actor encapsula estado local, comportamiento y una cola FIFO. Se comunica mediante **mensajes asíncronos e inmutables**, erradicando condiciones de carrera.



Let It Crash y Supervisión

Árboles de Supervisión jerárquicos con estrategias como *One-for-One* para reiniciar componentes de forma aislada.



Transparencia de Ubicación

La dirección de un actor es una abstracción pura: el sistema escala idénticamente en núcleos locales o nodos remotos.

CONCLUSIÓN

Evolución y Adaptación

Hemos explorado cómo los paradigmas de programación han evolucionado, cada uno ofreciendo soluciones innovadoras a los desafíos de la computación.

- El imperativo sentó las bases con control directo sobre el hardware.
- El POO introdujo la encapsulación y el polimorfismo para gestionar la complejidad.
- El funcional abrazó la inmutabilidad para la claridad y la concurrencia sin efectos secundarios.
- El lógico se enfocó en el "qué" en lugar del "cómo", para el razonamiento basado en reglas.
- El concurrente, especialmente el modelo de actores, aborda la escala y la resiliencia en sistemas distribuidos.



Hacia un Paradigma Unificado

La tendencia actual apunta a la **programación multiparadigma**, donde los desarrolladores combinan lo mejor de cada enfoque para construir sistemas más robustos, escalables y mantenibles.

Modularidad
Combinar POO y funcional para componentes claros.

Eficiencia
Control imperativo para optimizaciones de bajo nivel.



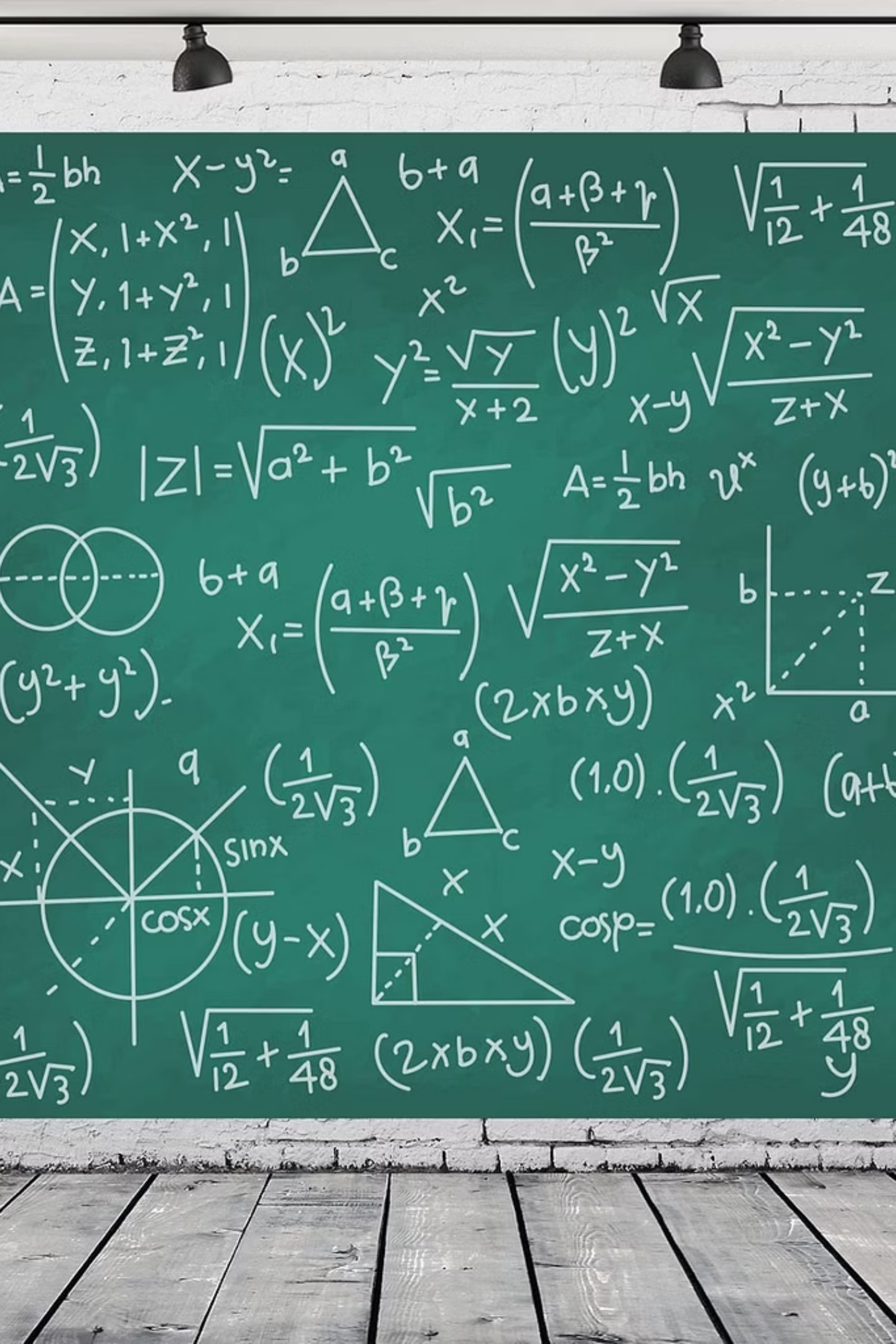
Concurrencia
Actores y funciones puras para sistemas distribuidos.

Inteligencia
Lógica declarativa para IA y sistemas expertos.

Análisis Léxico y Benchmarking

Zig · Rust · Python · JavaScript —
Conjetura de Collatz, $n < 100\ 000$





El Benchmark: Conjetura de Collatz

Algoritmo

Bifurcación condicional
densa + aritmética entera

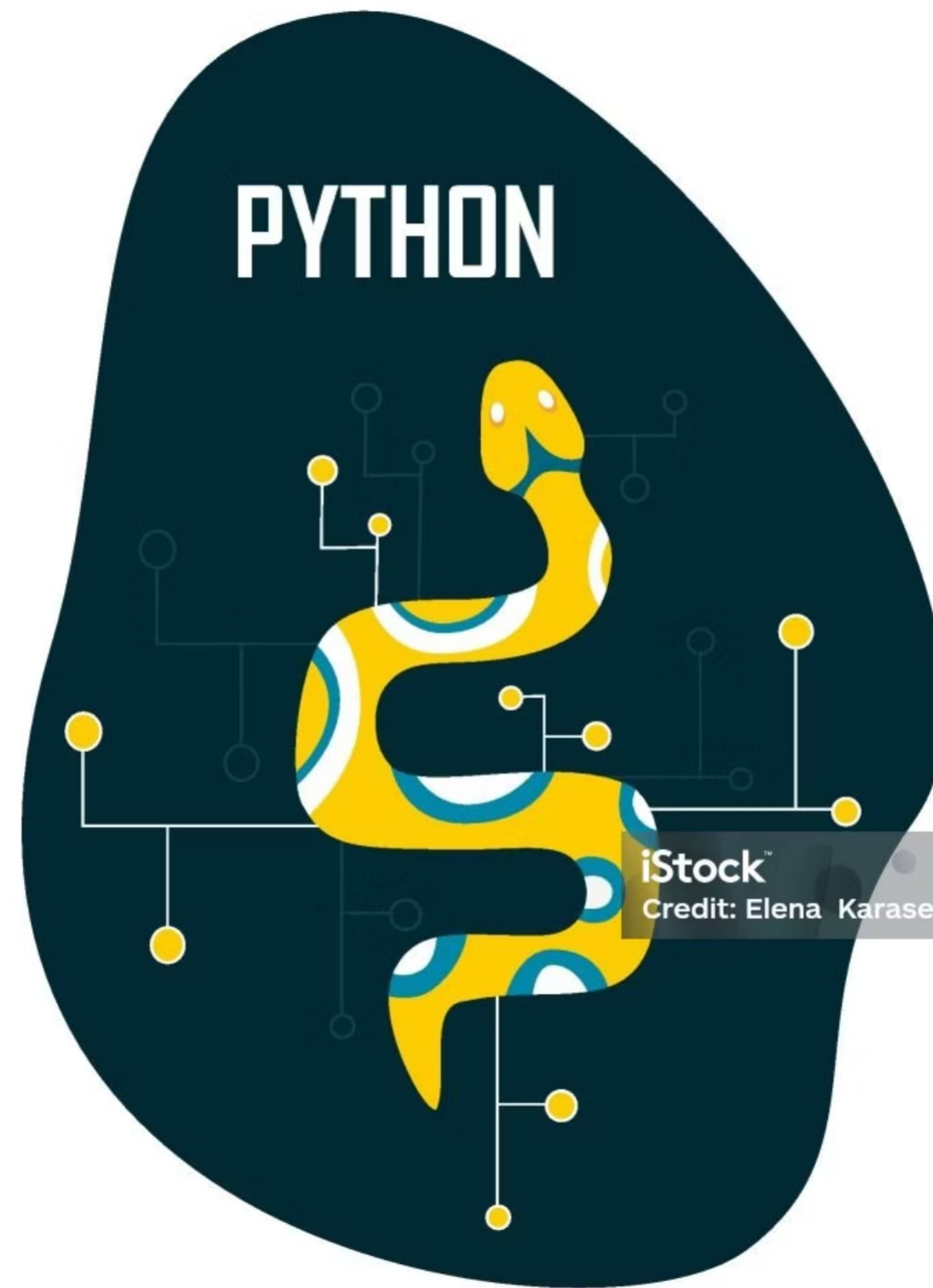
Rango

Todo $n < 100\,000$

Hardware

Intel Celeron 5205U · 2 núcleos · 4 GB RAM

Python — Visión General



1268535012

35 palabras reservadas

`def, class, async, await, with, yield...`

Identificadores

Soporta Unicode (PEP 3131) ·

`__dunder__` semántica especial ·

`snake_case` / `PascalCase`

Bloques

Indentación significativa —

`INDENT/DEDENT` son tokens explícitos

JavaScript — Visión General



ECMAScript 2024

`const, let, async, await, class, import...`

Identificadores

Soporta \$ y Unicode completo · ASI
(inserción automática de ;)

Ejecución

Motor **V8 JIT** — compilación
especulativa a código nativo

Zig — Visión General



Palabras reservadas

`const, var, fn, defer, comptime, anytype...`

Identificadores

`[a-zA-Z_][a-zA-Z0-9_]*` ·
Sensible a mayúsculas · `@"var"`
para colisiones

Bloques

Llaves `{}` · Punto y coma `;` · Sin GC
→ `defer` para liberar recursos

Rust — Visión General

Rust



Palabras reservadas

`fn, let, mut, match, trait, impl...` + 14
reservadas para uso futuro

Identificadores

Unicode NFKC · snake_case / PascalCase ·
advertencias por convención

Bloques

Llaves `{ }` · Última expresión sin `;` = retorno
implícito

Gramáticas BNF — Estructuras Esenciales

Zig

```
fn IDENT() Return Type Block ·  
if/while/for con bloques {}
```

Python

```
def IDENT(): Suite · Suite = NEWLINE  
INDENT...DEDENT
```

Rust

```
fn IDENT() -> Type Block · match Expr  
{ arm => }
```

JavaScript

```
function IDENT() Block · ( ) => Expr  
· ASI opcional
```




Resultados de Benchmarking

103ms

Rust 🏆

LLVM --release

131ms

Zig 🏆

LLVM ReleaseFast

463ms

JS 🏆

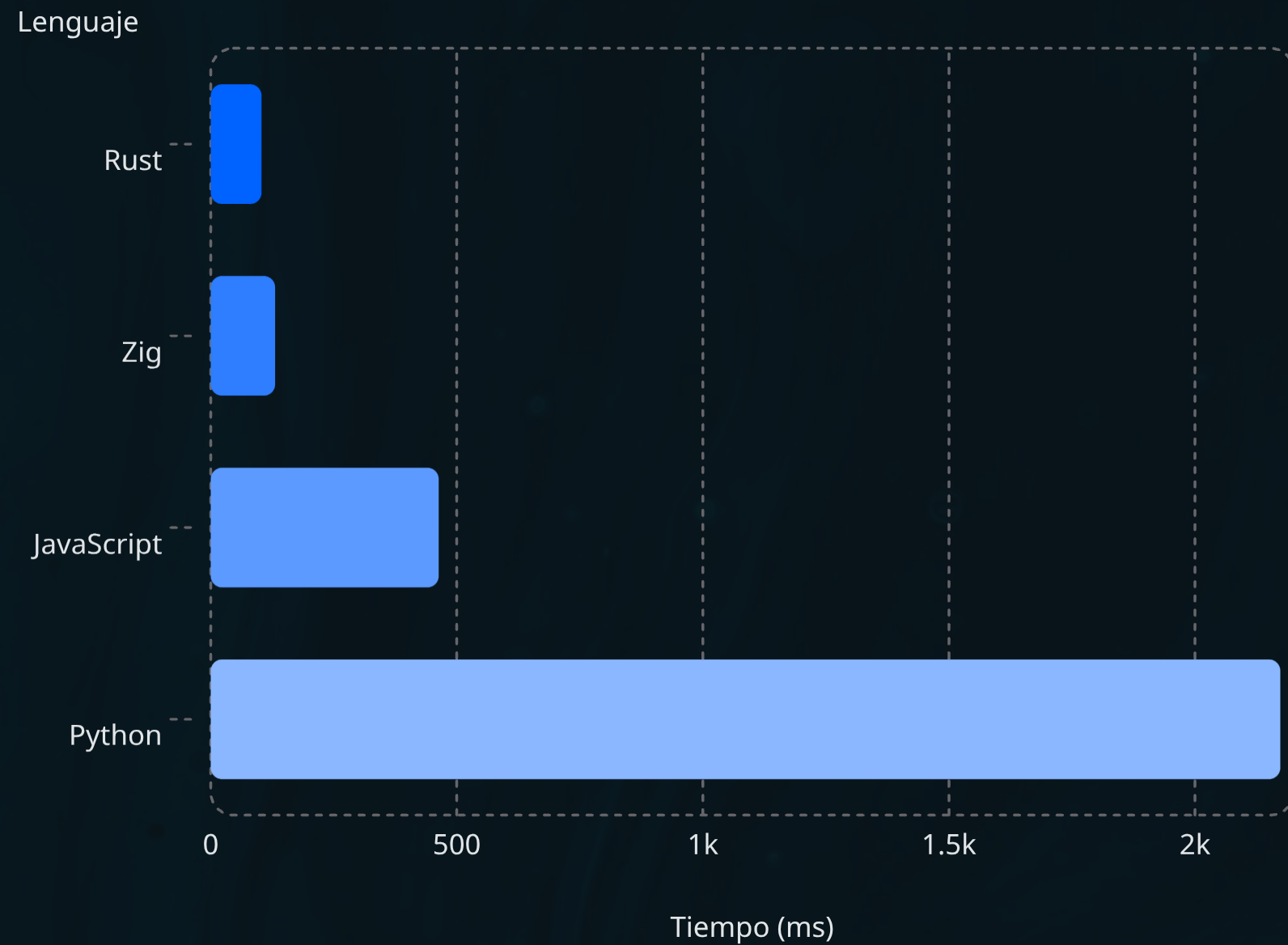
V8 JIT

2173ms

Python

CPython interpretado

Análisis de Resultados



Rust supera a Zig

Optimizaciones LLVM más agresivas para esta arquitectura



JS: resultado sólido

JIT eficiente, pero warm-up lento en hardware limitado



Python: último

Limitaciones de CPython y el GIL impiden paralelismo

Conclusión

Nativo > JIT >
Interpretado

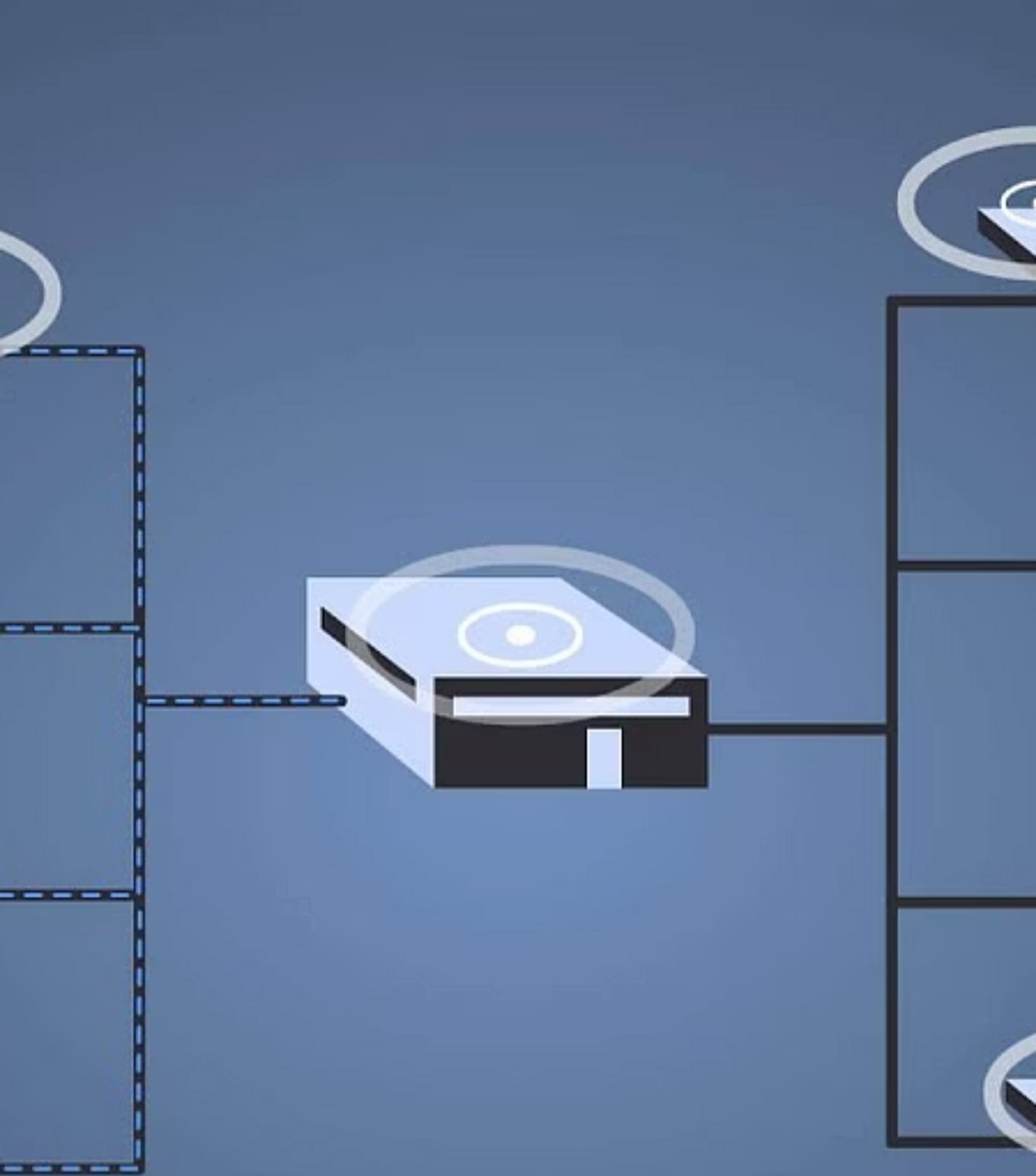
Jerarquía validada incluso
en hardware modesto

Contexto importa

Rust superó a Zig: el
rendimiento no es absoluto

Distancias se acortan

En Celeron, la brecha se reduce pero la ventaja compilada
persiste



ome management system would allow
communication between homes and the

Diseño de Lenguaje L (DSL)

Componentes del Alfabeto y Literales — Morfología Básica del
DSL para microredes inteligentes.

Alfabeto y Literales

Identificadores

Comienzan con letra minúscula; admiten letras, números y guiones bajos. Sin espacios.

`bateria_1`, `sensorFlujo`,
`refrigeracion`

Literales Numéricos

Enteros o decimales con punto

`.`
`55`, `20.5`, `-12`

Literales Booleanos

`correct` (alto) y `incorrect`
(bajo) para interruptores y relés.

Operadores y Delimitadores

Aritméticos

`+ - * ^ / // ++ --`

Relacionales

`> < == >= <= != === !==`

Lógicos

`and or inv` — `inv` invierte un estado lógico.

Delimitadores

`( ) { } , ; — ;` finaliza sentencia.

Comentarios

`#` Esto es un comentario — ignorados por el analizador léxico.

Primitivas Obligatorias

Diccionario esencial del DSL para microredes inteligentes.

Token	Tipo	Función
<code>init_grid</code>	Inicialización	Arranca la microred inteligente.
<code>leer_temperatura</code>	Sensor	Obtiene °C de un contenedor de baterías.
<code>estado_carga</code>	Sensor	Devuelve carga disponible (0%–100%).
<code>leer_potencia</code>	Sensor	Mide flujo eléctrico en kW de inversores o paneles.
<code>conmutar_linea</code>	Actuador	Conecta o aísla relés de un sector.

Estructuras de Control y Sistema

`if / or_if / otherwise`

Evalúan condiciones: `if` inicia el bloque, `or_if` evalúa alternativas y `otherwise` define el bloque por defecto si ninguna condición se cumple.

`while`

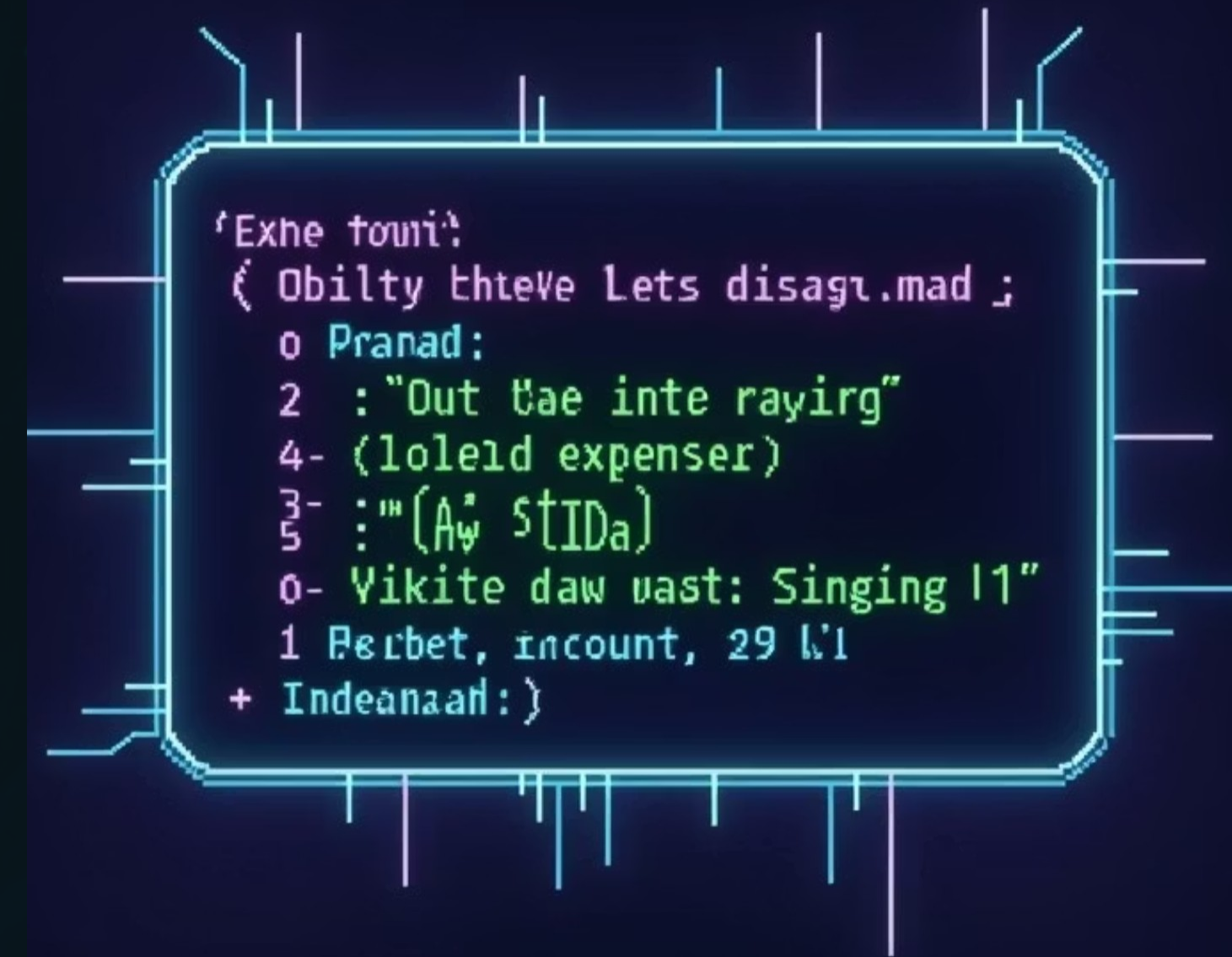
Inicia un bucle repetitivo basado en una condición lógica.

`{ y }`

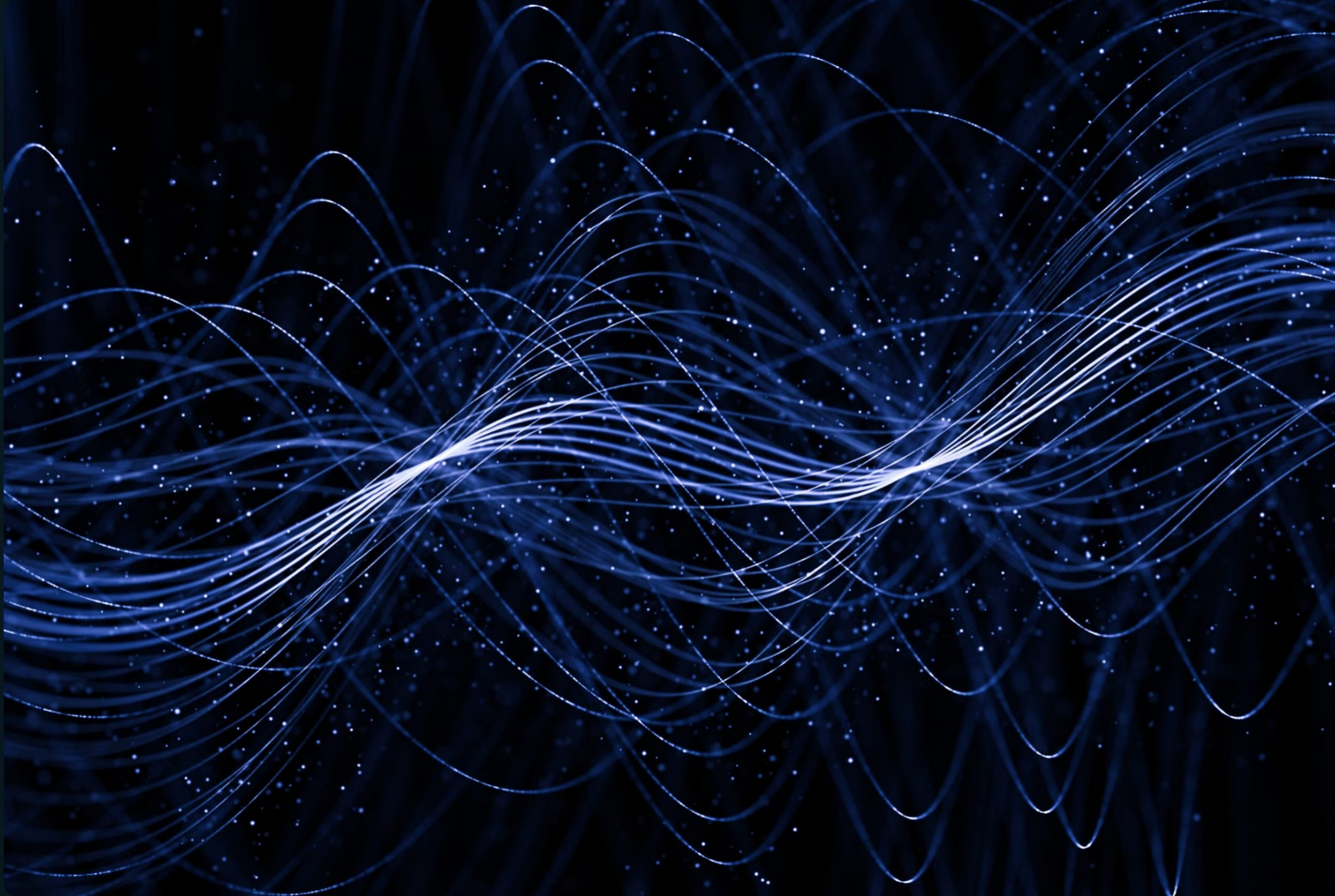
Delimitan apertura y cierre de bloques de instrucciones en `if`, `or_if`, `otherwise` y `while`.

`print`

Muestra mensajes o alertas en la interfaz Hombre-Máquina (HMI) del operador.



Gramática Sintáctica: Estructura Global



1 Regla de Inicialización

Todo script debe comenzar con `init_grid;`. Ninguna instrucción puede ejecutarse antes de esta sincronización de hardware.

2 Regla de Secuenciación

Las sentencias se ejecutan de arriba hacia abajo. Cada instrucción operativa, asignación o lectura de sensor debe finalizar con `;`.

Asignación y Expresiones Matemáticas

1

Estructura de Asignación

```
[Identificador] =  
[Expresión o Sensor];
```

2

Precedencia Aritmética

1.º ^ → 2.º * / → 3.º + -. Los paréntesis () rompen la precedencia.

3

Operadores Unarios ++ / --

Se colocan tras el identificador (ciclos++). Máxima prioridad de evaluación.

Flujo de Control e Inversión Lógica

Condicional Completo

`if` + condición → `{ }`. Opciones con `or_if` y bloque por defecto con `otherwise`.

Bucle `while`

`while` + condición (o `correct` para bucle infinito) → bloque `{ }`.

Operador `inv`

Máxima prioridad en expresiones booleanas. Se coloca antes de un identificador de estado o sensor booleano.

Transforma `correct` en `incorrect` y viceversa.

Prevención de Fuga Térmica

```
init_grid;
while correct {
  tempBatPrincipal = leer_temperatura(bat_01);
  tempBatRespaldo = leer_temperatura(bat_02);
  if tempBatPrincipal > 55 {
    conmutar_linea(refrigeracion_aux, correct);
    conmutar_linea(carga_solar, incorrect);
    conmutar_linea(sector_industrial, incorrect);
    conmutar_linea(red_comercial, correct);
    print("ALERTA: Fuga térmica detectada.");
  }
  or_if temp_respaldo > 45 {
    conmutar_linea(refrigeracion_aux2, correct);
    print("PRECAUCIÓN: Batería de respaldo caliente.");
  }
  otherwise {
    if inv temp_principal > 55 {
      conmutar_linea(refrigeracion_aux, incorrect);
    }
  }
}
```

Protocolo de Seguridad

Monitoreo térmico continuo de **bat_01** y **bat_02** en bucle infinito.

- Si temp. principal > 55 °C:
activa refrigeración auxiliar,
detiene carga solar, aísla sector
industrial y desvía consumo a
red comercial.
- Si temp. respaldo > 45 °C:
activa enfriamiento preventivo
del respaldo.
- En estado normal: desactiva
extractores de emergencia
mediante **inv**.

Balance de Carga y Venta de Energía

```
init_grid;
carga_actual = estado_carga(bat_01);
generacion_solar = leer_potencia(paneles_sector_A);
demanda_interna = leer_potencia(inversor_principal);

if carga_actual > 90 and generacion_solar > demanda_interna {
    conmutar_linea(reles_automaticos, correct);
    print("Inyectando excedente a la red pública.");
}
or_if carga_actual < 20 {
    conmutar_linea(sectores_no_esenciales, incorrect);
    conmutar_linea(sector_medico, correct);
    conmutar_linea(sector_servidores, correct);
    print("AVISO: Carga crítica. Sectores no esenciales aislados");
}
otherwise {
    print("Sistema operando en rangos estables.");
}
```

Gestión Inteligente de Energía

- **Caso 1 — Excedente:** Si carga > 90% y generación solar supera la demanda, inyecta excedente a la red pública.
- **Caso 2 — Carga crítica:** Si carga < 20%, aísla sectores no esenciales y preserva áreas médicas y servidores.
- **Caso 3 — Normal:** Operación estable sin intervención.

¡Gracias!

